DBMS Lab Viva & Complete Syntax Guide

WEEKS 1 TO 8: TOPIC-WISE VIVA Q&A + COMPLETE SQL & PL/SQL SYNTAX REFERENCE

Course: DBMS Lab (IARE)
Coverage: Exercises 1–8 Complete

TOPIC 1 SQL Command Categories (DDL, DML, DCL, TCL) — Viva Q&A

Q1. What are the 4 main categories of SQL commands?
1. DDL (Data Definition Language): Defines and modifies table structure (`CREATE` , `ALTER` , `DROP` , `TRUNCATE` , `RENAME`). All DDL commands are auto-committed.
2. DML (Data Manipulation Language): Inserts, updates, deletes, and retrieves records inside tables (`INSERT` , `UPDATE` , `DELETE` , `SELECT`).
3. DCL (Data Control Language): Controls user access and database privileges (`GRANT` , `REVOKE`).
4. TCL (Transaction Control Language): Manages changes made by DML statements (`COMMIT` , `ROLLBACK` , `SAVEPOINT`).

Q2. What is the use of the ALTER command in SQL?
`ALTER` is a DDL command used to modify the structure of an existing table without deleting it. It is used to: (1) `ADD` a new column or constraint, (2) `MODIFY` an existing column's data type or size, (3) `DROP COLUMN` or `DROP CONSTRAINT` , and (4) `RENAME COLUMN` .

Q3. How do you check the structure of a table and format output in Oracle SQL*Plus?
Use `DESC table_name;` (or `DESCRIBE`) to view column names, data types, and `NOT NULL` constraints. Use `SET LINESIZE 200;` and `SET PAGESIZE 100;` to prevent row wrapping in SQL*Plus.

TOPIC 2 DELETE vs. TRUNCATE vs. DROP & TCL Commands

Feature	DELETE	TRUNCATE	DROP
Category	DML (Data Manipulation)	DDL (Data Definition)	DDL (Data Definition)
What it removes	Removes selected rows or all rows	Removes all rows; keeps table structure	Removes all rows AND table structure
WHERE Clause	Supported (e.g., `WHERE id=10`)	Not supported	Not supported
Rollback	Can be undone using `ROLLBACK`	Cannot be rolled back (auto-commits)	Cannot be rolled back (auto-commits)
Speed & Triggers	Slower (row-by-row); fires triggers	Very fast; does not fire triggers	Instant; drops table from schema

Viva Question — Difference between COMMIT, ROLLBACK, and SAVEPOINT:

- `COMMIT` : Permanently saves all DML changes made in the current transaction to the database.
- `ROLLBACK` : Undoes uncommitted DML changes back to the last `COMMIT` or to a named `SAVEPOINT` .
- `SAVEPOINT s1` : Creates a checkpoint inside a transaction so you can partially undo changes using `ROLLBACK TO s1;` without losing earlier work.

TOPIC 3 Database Constraints, Keys & DCL Users — Viva Q&A

Q4. What is the difference between PRIMARY KEY, UNIQUE, and NOT NULL constraints?
• **PRIMARY KEY:** Uniquely identifies each row. It does **not** allow `NULL` values and does **not** allow duplicates. A table can have only **one** Primary Key (which can be single-column or a composite key like `PRIMARY KEY(SID, BID)`).
• **UNIQUE:** Ensures all values in a column are distinct (no duplicates), but it **allows NULL values**. A table can have multiple `UNIQUE` constraints.
• **NOT NULL:** Ensures a column cannot store `NULL` (empty) values, though duplicate values are allowed.

Q5. What is a FOREIGN KEY and Referential Integrity?
A **Foreign Key** is a column in a child table (e.g., `EMP.DEPTNO`) that references the **Primary Key** of a parent table (e.g., `DEPT.DEPTNO`). It enforces **referential integrity**, meaning you cannot insert a `DEPTNO` in `EMP` that does not exist in `DEPT`, and you cannot delete a parent record if child records depend on it.

Q6. How do you create a User and grant permissions in Oracle?
Logged in as `SYSTEM`: (1) `CREATE USER user1 IDENTIFIED BY user123;` creates the account. (2) `GRANT CREATE SESSION, RESOURCE TO user1;` allows the user to log in (`CREATE SESSION`) and create tables/objects (`RESOURCE`). (3) `GRANT ALL ON Employee TO user1;` gives `SELECT`, `INSERT`, `UPDATE`, and `DELETE` privileges on the table.

TOPIC 4 Aggregate Functions, GROUP BY & Built-In Functions — Viva Q&A

Function / Clause	Purpose in SQL	Lab Manual Example
`COUNT(*)` / `COUNT(col)`	Counts total rows (`COUNT(*)` includes NULLs; `COUNT(col)` ignores NULLs)	`SELECT COUNT(*) FROM EMP;`
`SUM()` & `AVG()`	Calculates total sum and arithmetic average of a numeric column	`SELECT AVG(SAL) FROM EMP;`
`MIN()` & `MAX()`	Returns lowest and highest value in a column	`SELECT MIN(SAL), MAX(SAL) FROM EMP;`
`SUBSTR(str, m, n)`	Extracts `n` characters starting from position `m`	`SUBSTR('DATABASE', 1, 4)` → `'DATA'`
`ADD_MONTHS(d, n)`	Adds `n` calendar months to date `d`	`ADD_MONTHS(ORDERDATE, 6)`
`NVL(expr1, expr2)`	Replaces `NULL` value with a default value (`expr2`)	`NVL(SUM(netsalary), 0)`

Q7. What is the difference between WHERE and HAVING clauses?
`WHERE` filters individual rows **before** grouping and cannot contain aggregate functions. `HAVING` filters grouped results **after** the `GROUP BY` clause and is used with aggregate functions (e.g., `GROUP BY DEPTNO HAVING COUNT(*) > 3`).

Q8. What is the DUAL table in Oracle?
`DUAL` is a built-in dummy 1-row, 1-column table in Oracle used to evaluate expressions, built-in functions, or call user-defined functions when no user table is required (e.g., `SELECT SIGN(100), SYSDATE FROM DUAL;`).

Q9. What is the use of LIKE, wildcard characters (% and _), MOD, and TRUNC?
`LIKE` is used for pattern matching: `%` matches zero or more characters (e.g., `LIKE 'B%B'` matches names starting and ending with B) and `_` matches a single character. `MOD(n, 10)` returns the remainder (extracts last digit), and `TRUNC(n/10)` removes the decimal part (removes last digit).

TOPIC 5 Joins, Subqueries & Relational Division — Viva Q&A

Q10. What is a JOIN and what is the difference between INNER JOIN and LEFT JOIN?
A `JOIN` combines columns from two or more tables based on a related column (Foreign Key = Primary Key).
• **INNER JOIN (or JOIN):** Returns only the rows that have matching values in both tables.
• **LEFT JOIN:** Returns all rows from the left table (e.g., `DEPT`) and matching rows from the right table (`EMP`). If a department has 0 employees, it still appears with count `0`.

Q11. What is a Subquery and a Correlated Subquery?
A **Subquery** is a query nested inside another SQL query's `WHERE` or `HAVING` clause (e.g., finding the second maximum cost using `WHERE COST < (SELECT MAX(COST) FROM PART)`).
A **Correlated Subquery** is an inner query that depends on the outer query and executes once for each outer row (e.g., `WHERE SAL = (SELECT MIN(SAL) FROM EMP WHERE DEPTNO = E.DEPTNO)` to find the lowest paid employee in each department).

Q12. How does NOT EXISTS work for "Reserved ALL Boats" (Division Query)?
SQL does not have a direct `FOR ALL` operator, so double `NOT EXISTS` is used: "Find sailors such that there does `NOT EXIST` an INTERLAKE boat that the sailor has `NOT` reserved."

TOPIC 6 PL/SQL Block Structure, Loops & Algorithms — Viva Q&A

Q13. What is PL/SQL and what is its basic block structure?
PL/SQL (Procedural Language/SQL) is Oracle's procedural extension to SQL that lets you use variables, loops, `IF-THEN-ELSE` conditions, and exception handling. A PL/SQL block has 3 sections:
1. `DECLARE` (Optional): Declares variables, constants, and local subprograms.
2. `BEGIN` (Mandatory): Contains executable SQL and procedural statements.
3. `EXCEPTION` (Optional): Handles runtime errors.
4. `END; /` (Mandatory): Closes the block and `/` executes it.

Q14. What is the purpose of SET SERVEROUTPUT ON and DBMS_OUTPUT.PUT_LINE?
By default, SQL*Plus does not display PL/SQL output on screen. `SET SERVEROUTPUT ON;` enables the output buffer so that `DBMS_OUTPUT.PUT_LINE(...)` can print results on the console. `DBMS_OUTPUT.PUT(...)` prints on the same line without a newline, while `DBMS_OUTPUT.NEW_LINE;` breaks the line.

Q15. Explain the operators and loops used in PL/SQL.
• **Assignment Operator (`:=`):** Assigns a value to a variable (e.g., `fact := 1;`).
• **Concatenation Operator (`||`):** Joins strings and variables for printing (e.g., `'Sum = ' || v_sum`).
• **WHILE Loop:** `WHILE condition LOOP ... END LOOP;` repeats while condition is true (used when number of digits is unknown, like Reverse, Palindrome, Sum of Digits, GCD).
• **FOR Loop:** `FOR i IN 1..n LOOP ... END LOOP;` automatically increments counter `i` from `1` to `n` (used in Factorial, Prime Check, Circle Area from radius 3 to 7).

> **Quick Logic Formulas for Lab Viva (Weeks 4, 5, 7, 8):**
> - **Reverse / Palindrome / Sum of Digits:** Extract last digit: `d := MOD(n, 10);` | Build reverse: `rev := rev * 10 + d;` | Add digit: `sum := sum + d;` | Shrink number: `n := TRUNC(n / 10);`. If `rev = temp`, it is a **Palindrome**.
> - **Prime Number:** Count divisors from `1..n` where `MOD(n, i) = 0`. If `count = 2`, it is Prime.
> - **Perfect Number:** Sum proper divisors from `1..(n-1)` where `MOD(n, i) = 0`. If `sum = n`, it is Perfect (e.g., `6 = 1+2+3`, `28`).
> - **GCD (Euclidean Algorithm):** Repeat `r := MOD(x, y); x := y; y := r;` while `y != 0`. Final `x` is GCD.

TOPIC 7 Stored Procedures vs. Functions (Weeks 5, 7, 8) — Viva Q&A

Feature	Stored Procedure	Function
Return Value	May or may not return a value	**Must** return a single value using `RETURN`
RETURN Keyword	No `RETURN datatype` in header	Header must include `RETURN datatype`
Call inside SELECT	Cannot be called inside a `SELECT` query	Can be called directly inside `SELECT ... FROM DUAL;`
Execution Syntax	`EXEC proc_name(params);` or inside `BEGIN ... END;`	`SELECT func_name(params) FROM DUAL;` or assigned to a variable
Main Purpose	Used to perform an action/task (e.g., `PALINDROME`, `GCD`, `LUCKY_NUMBER`)	Used to compute and return a result (e.g., `factorial(n)`, `get_basic_hra(101)`)

Q16. What is a Local Function vs. a Stored Function? How do you check compilation errors?
A **Stored Function** is created with `CREATE OR REPLACE FUNCTION` and saved permanently in the database schema. A **Local Function** (like `FUNCTION is_prime(n NUMBER) RETURN BOOLEAN` in Week 5) is declared inside the `DECLARE` section of an anonymous block and exists only during that block's execution. To view syntax errors in a stored procedure/function, run `SHOW ERRORS FUNCTION func_name;`.

Q17. What does SELECT ... INTO do inside a PL/SQL Function?
In PL/SQL, `SELECT col INTO variable_name FROM table WHERE ...` fetches a value from a database table and stores it inside a PL/SQL variable (e.g., `SELECT basic + hra INTO v_total FROM employee WHERE empno = p_empno;`).

TOPIC 8 Database Triggers (Week 6) — Viva Q&A

Q18. What is a Trigger in DBMS and when is it used?
A **Trigger** is a stored PL/SQL block associated with a table that **fires (executes) automatically** when a specified DML event (`INSERT`, `UPDATE`, or `DELETE`) occurs on that table. It is used for data validation (checking 6-digit passport ID), automatic formatting (converting employee name to `UPPER`), backup of deleted rows, and transparent auditing.

Q19. What is the difference between BEFORE and AFTER triggers, and Row-Level vs. Statement-Level triggers?
• **BEFORE Trigger:** Fires *before* the data is written to the table. Used to validate data (`RAISE_APPLICATION_ERROR`) or modify incoming column values (`:NEW.EMP_NAME := UPPER(:NEW.EMP_NAME)`).
• **AFTER Trigger:** Fires *after* the DML change has completed. Used to print messages or insert audit records into an audit table.
• **Row-Level Trigger (`FOR EACH ROW`):** Executes once for every individual row affected by the DML statement and gives access to `:OLD` and `:NEW` pseudo-records. Without `FOR EACH ROW`, it is a Statement-Level trigger that executes only once per SQL statement.

Q20. Explain :NEW, :OLD, INSERTING, UPDATING, DELETING, and RAISE_APPLICATION_ERROR.
• `:NEW.col` holds the new value being inserted or updated. `:OLD.col` holds the previous value before an update or delete (for `INSERT`, `:OLD` is `NULL`; for `DELETE`, `:NEW` is `NULL`).
• `INSERTING`, `UPDATING`, and `DELETING` are conditional predicates used inside a combined trigger (`AFTER INSERT OR UPDATE OR DELETE`) to detect which operation fired the trigger.
• `RAISE_APPLICATION_ERROR(-20001, 'Message')` raises a custom user-defined error (error code between `-20000` and `-20999`) and aborts/rolls back the invalid DML statement.

SYNTAX MASTER 1 All DDL & Constraint Commands (Syntax + Lab Examples)

1. CREATE TABLE (Basic & With Constraints)

```
-- Syntax:
CREATE TABLE table_name (
    col1 datatype PRIMARY KEY,
    col2 datatype NOT NULL,
    col3 datatype,
    CONSTRAINT fk_name FOREIGN KEY (col3)
        REFERENCES parent_table(pk_col)
);

-- Lab Example (EMP & RESERVES Composite
PK):
CREATE TABLE EMP (
    EMPNO NUMBER(4) PRIMARY KEY,
    ENAME VARCHAR2(20) NOT NULL,
    SAL NUMBER(8,2),
    DEPTNO NUMBER(2),
    CONSTRAINT FK_DEPT FOREIGN KEY (DEPTNO)
        REFERENCES DEPT(DEPTNO)
);
CREATE TABLE RESERVES (
    SID NUMBER, BID NUMBER, DAY DATE,
    PRIMARY KEY (SID, BID),
    FOREIGN KEY (SID) REFERENCES
SAILORS(SID),
    FOREIGN KEY (BID) REFERENCES BOATS(BID)
);
```

2. DROP, TRUNCATE, RENAME & DESC

```
-- Drop Table:
Syntax:  DROP TABLE table_name;
Example: DROP TABLE Branch;

-- Truncate Table:
Syntax:  TRUNCATE TABLE table_name;
Example: TRUNCATE TABLE Employee;

-- Rename Table:
Syntax:  RENAME old_name TO new_name;
Example: RENAME Emp TO Employee_New;

-- Describe Structure:
Syntax:  DESC table_name;
Example: DESC Employee;
```

3. ALTER TABLE (All 8 Variations in Lab PDFs)

```
-- A. Add a New Column:
Syntax:  ALTER TABLE tbl ADD col datatype;
Example: ALTER TABLE Employee ADD Commission
NUMBER;

-- B. Modify Column Data Type / Size:
Syntax:  ALTER TABLE tbl MODIFY col
new_datatype;
Example: ALTER TABLE Branch MODIFY Assets
NUMBER(12,2);

-- C. Add NOT NULL Constraint to Column:
Syntax:  ALTER TABLE tbl MODIFY col datatype
NOT NULL;
Example: ALTER TABLE EMPLOYEE
         MODIFY ENAME VARCHAR2(20) NOT NULL;

-- D. Rename a Column:
Syntax:  ALTER TABLE tbl RENAME COLUMN old_c
TO new_c;
Example: ALTER TABLE Employee
         RENAME COLUMN Sal TO Salary;

-- E. Drop a Column:
Syntax:  ALTER TABLE tbl DROP COLUMN
col_name;
Example: ALTER TABLE Reserves DROP COLUMN
Time;

-- F. Add PRIMARY KEY Constraint:
Syntax:  ALTER TABLE tbl ADD CONSTRAINT
con_name
         PRIMARY KEY (col);
Example: ALTER TABLE EMPLOYEE
         ADD CONSTRAINT emp_pk PRIMARY KEY
(EMPNO);

-- G. Add UNIQUE Constraint:
Syntax:  ALTER TABLE tbl ADD CONSTRAINT
con_name
         UNIQUE (col);
Example: ALTER TABLE DEPARTMENT
         ADD CONSTRAINT dept_unique UNIQUE
(DEPTNAME);

-- H. Add FOREIGN KEY Constraint:
Syntax:  ALTER TABLE child_tbl ADD
CONSTRAINT fk_name
         FOREIGN KEY (col) REFERENCES
parent_tbl(col);
Example: ALTER TABLE EMPLOYEE ADD CONSTRAINT
emp_dept_fk
         FOREIGN KEY (DEPTNO)
         REFERENCES DEPARTMENT(DEPTNO);
```

SYNTAX MASTER 2 All DML, DCL & TCL Commands (Syntax + Lab Examples)

1. INSERT, UPDATE & DELETE (DML)

```
-- INSERT All Columns:
Syntax:  INSERT INTO tbl VALUES (v1, v2,
v3);
```

2. DCL (User & Grant) & TCL (Transactions)

```
-- Create User & Grant Permissions (DCL):
Syntax:  CREATE USER username IDENTIFIED BY
password;
```

SYNTAX MASTER 3 SELECT, Aggregate, GROUP BY & Built-In Functions

1. SELECT, WHERE, LIKE, ORDER BY & DISTINCT

```
-- General SELECT Syntax:
SELECT DISTINCT col1, col2, expr AS alias
FROM table_name
WHERE condition
ORDER BY col1 ASC | DESC;

-- Lab Examples:
-- Pattern Matching (Starts with B or C):
SELECT * FROM EMP
WHERE ENAME LIKE 'B%' OR ENAME LIKE 'C%';

-- Begins & Ends with B and length >= 3:
SELECT SNAME, AGE FROM SAILORS
WHERE SNAME LIKE 'B%B' AND LENGTH(SNAME) >=
3;

-- Calculated Column (15% Salary Increment):
SELECT ENAME, SAL, SAL*0.15 AS
SAL_INCREMENT,
       SAL*1.15 AS NEW_SALARY FROM EMP;

-- Sorting Ascending / Descending:
SELECT ENAME FROM EMP ORDER BY ENAME DESC;
SELECT EMPNO, ENAME FROM EMP ORDER BY EMPNO
ASC;
```

2. Aggregate Functions, GROUP BY & HAVING

```
-- Syntax:
SELECT group_col, COUNT(*), MIN(col),
MAX(col),
       SUM(col), AVG(col)
FROM table_name
WHERE row_condition
GROUP BY group_col
HAVING group_condition;

-- Lab Examples:
-- Average Salary per Job excluding Manager:
SELECT JOB, AVG(SAL) AS AVG_SALARY FROM EMP
WHERE JOB != 'MANAGER' GROUP BY JOB;

-- Departments with more than 3 employees:
SELECT DEPTNO, AVG(SAL) FROM EMP
GROUP BY DEPTNO HAVING COUNT(*) > 3;

-- Youngest voting-eligible sailor per
rating (>=2 sailors):
SELECT RATING, MIN(AGE) AS YOUNGEST_AGE
FROM SAILORS WHERE AGE >= 18
GROUP BY RATING HAVING COUNT(*) >= 2;
```

3. Built-In String, Numeric & Date Functions (From Exercise 1, 3, 5)

```
-- SIGN(n), SUBSTR(str, start, len), CONCAT(s1, s2):
SELECT SIGN(100) AS POS, SIGN(0) AS ZERO, SIGN(-100) AS NEG FROM DUAL;
SELECT CONCAT(SUBSTR('DATABASE', 1, 4), SUBSTR('MANAGEMENT', 1, 4)) AS RESULT FROM DUAL;

-- SYSDATE, TRUNC, ADD_MONTHS, EXTRACT, TO_NUMBER, TO_CHAR, TO_DATE, NVL:
SELECT EMPNO, ENAME, DOB, TRUNC(SYSDATE - DOB) AS DAYS_LIVED FROM EMP;
SELECT * FROM SUPPLY WHERE DELIVERYDATE <= ADD_MONTHS(ORDERDATE, 6);
SELECT NVL(SUM(netsalary), 0) FROM employee WHERE EXTRACT(YEAR FROM saldate) = 2025;
DELETE FROM Sailor WHERE TO_NUMBER(Rating) > 8;
```

SYNTAX MASTER 4 Joins, Subqueries & Relational Division Queries

1. INNER JOIN, LEFT JOIN & 3-Table JOIN

```
-- LEFT JOIN (Department Employee Count):
SELECT D.DEPTNO, D.DNAME, COUNT(E.EMPNO) AS
TOTAL
FROM DEPT D LEFT JOIN EMP E ON D.DEPTNO =
E.DEPTNO
GROUP BY D.DEPTNO, D.DNAME;

-- 3-Table JOIN (Sailors who reserved RED
boat):
SELECT DISTINCT S.SNAME
FROM SAILORS S
JOIN RESERVES R ON S.SID = R.SID
JOIN BOATS B ON R.BID = B.BID
WHERE B.COLOR = 'RED'
ORDER BY S.SNAME;

-- Vendors supplying BOTH Assembled and Sub
parts:
SELECT V.VID, V.VNAME FROM VENDOR V
```

2. Subqueries, Correlated & Division (NOT EXISTS)

```
-- Correlated Subquery (Lowest paid per
Dept):
SELECT * FROM EMP E
WHERE SAL = (SELECT MIN(SAL) FROM EMP
             WHERE DEPTNO = E.DEPTNO);

-- Second Maximum Cost Assembled Part:
SELECT PID, PNAME, COST FROM PART
WHERE PTYPE = 'Assembled' AND COST = (
    SELECT MAX(COST) FROM PART
    WHERE PTYPE = 'Assembled' AND COST < (
        SELECT MAX(COST) FROM PART
        WHERE PTYPE = 'Assembled'
    )
);

-- Division Query (Reserved ALL 'INTERLAKE'
boats):
SELECT S.SID, S.SNAME FROM SAILORS S
```

SYNTAX MASTER 5 PL/SQL Anonymous Blocks, Control Structures & Loops (Week 4)

1. Anonymous Block & IF-ELSIF-ELSE Syntax

```
SET SERVEROUTPUT ON;
DECLARE
    a NUMBER := 10; b NUMBER := 25; c NUMBER
:= 15;
BEGIN
    IF a >= b AND a >= c THEN
        DBMS_OUTPUT.PUT_LINE('Largest = ' ||
a);
    ELSIF b >= a AND b >= c THEN
        DBMS_OUTPUT.PUT_LINE('Largest = ' ||
b);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Largest = ' ||
c);
    END IF;
END;
/
```

2. WHILE Loop (Reverse & Sum of Digits)

```
SET SERVEROUTPUT ON;
DECLARE
    num NUMBER := 12345; digit NUMBER;
    rev NUMBER := 0; sum_d NUMBER := 0;
BEGIN
    WHILE num > 0 LOOP
        digit := MOD(num, 10);
        sum_d := sum_d + digit;
        rev   := rev * 10 + digit;
        num   := TRUNC(num / 10);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Sum=' || sum_d ||
' Rev=' || rev);
END;
/
```

3. FOR Loop (Prime Check, Factorial & Circle Area)

```
-- Prime Number & Factorial using FOR Loop:
SET SERVEROUTPUT ON;
DECLARE
    num NUMBER := 5; fact NUMBER := 1; cnt
NUMBER := 0;
BEGIN
    FOR i IN 1..num LOOP
        fact := fact * i;
        IF MOD(num, i) = 0 THEN cnt := cnt +
1; END IF;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Factorial = ' ||
fact);
    IF cnt = 2 THEN
        DBMS_OUTPUT.PUT_LINE(num || ' is
Prime');
    END IF;
END;
/

-- Loop Inserting into Table (Circle Area r
= 3..7):
DECLARE
    v_area NUMBER(10,2);
BEGIN
    FOR r IN 3..7 LOOP
        v_area := 3.14159 * r * r;
        INSERT INTO areas VALUES (r,
v_area);
    END LOOP;
    COMMIT;
END;
/
```

SYNTAX MASTER 6 PL/SQL Stored Functions & Procedures (Weeks 5, 7, 8)

1. Stored Function Syntax & Execution

```
-- Syntax & Table Lookup Function (Basic +
HRA):
CREATE OR REPLACE FUNCTION
get_basic_hra(p_empno NUMBER)
RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT basic + hra INTO v_total
    FROM employee WHERE empno = p_empno;
    RETURN v_total;
END;
/
-- Execute Function via DUAL or PL/SQL
Block:
SELECT get_basic_hra(101) AS "BASIC+HRA"
FROM dual;

-- Factorial Function & NCR Calculation:
CREATE OR REPLACE FUNCTION factorial(n
NUMBER)
RETURN NUMBER IS
    fact NUMBER := 1;
```

2. Stored Procedure Syntax & Execution

```
-- Syntax (Palindrome / Perfect / GCD /
Fibonacci):
CREATE OR REPLACE PROCEDURE proc_name(p_in
IN NUMBER)
IS
    -- local variable declarations
BEGIN
    -- procedural statements
END;
/
-- Execute Procedure:
EXEC proc_name(121);
-- OR: BEGIN proc_name(121); END; /

-- GCD Euclidean Procedure Example:
CREATE OR REPLACE PROCEDURE GCD(A IN NUMBER,
B IN NUMBER)
IS
    X NUMBER := A; Y NUMBER := B; R NUMBER;
BEGIN
    IF X < Y THEN R := X; X := Y; Y := R;
END IF;
    WHILE Y != 0 LOOP
```

SYNTAX MASTER 7 All PL/SQL Database Triggers (Week 6 Complete Syntax & Examples)

1. General Trigger Syntax & Salary Diff Trigger

```
-- General Trigger Syntax:
CREATE OR REPLACE TRIGGER trigger_name
BEFORE | AFTER
INSERT OR UPDATE OR DELETE ON table_name
FOR EACH ROW
BEGIN
    IF INSERTING THEN ...
    ELSIF UPDATING THEN ...
    ELSIF DELETING THEN ...
    END IF;
END;
/

-- Ex 6.1: Row-Level Salary Difference
Trigger:
CREATE OR REPLACE TRIGGER
customer_salary_trigger
AFTER INSERT OR UPDATE OR DELETE ON
CUSTOMERS
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        DBMS_OUTPUT.PUT_LINE('New Sal: ' ||
:NEW.SALARY);
        DBMS_OUTPUT.PUT_LINE('Diff: ' ||
            (NVL(:NEW.SALARY,0) -
NVL(:OLD.SALARY,0)));
    ELSIF UPDATING THEN
        DBMS_OUTPUT.PUT_LINE('Old: ' ||
:OLD.SALARY ||
            ' New: ' || :NEW.SALARY || '
Diff: ' ||
            (NVL(:NEW.SALARY,0) -
NVL(:OLD.SALARY,0)));
    ELSIF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('Deleted: ' ||
:OLD.NAME);
    END IF;
END;
/
```

2. Validation Trigger (6-Digit Passport Check)

```
CREATE OR REPLACE TRIGGER check_passport_id
BEFORE INSERT ON Passenger
FOR EACH ROW
BEGIN
    IF LENGTH(TO_CHAR(:NEW.Passport_id)) !=
6 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Passport ID must contain
exactly 6 digits'
        );
    END IF;
END;
/
```

3. Uppercase Formatting Trigger (BEFORE INSERT/UPDATE)

```
CREATE OR REPLACE TRIGGER emp_name_uppercase
BEFORE INSERT OR UPDATE ON EMPLOYEE
FOR EACH ROW
BEGIN
    :NEW.EMP_NAME := UPPER(:NEW.EMP_NAME);
END;
/
```

4. Backup Deleted Row Trigger (BEFORE DELETE)

```
CREATE OR REPLACE TRIGGER emp_delete_backup
BEFORE DELETE ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO DELETE_EMP (
        EMP_ID, EMP_NAME, AGE, DEPT,
        SALARY, DELETED_BY, DELETED_DATE
    ) VALUES (
        :OLD.EMP_ID, :OLD.EMP_NAME,
:OLD.AGE, :OLD.DEPT,
        :OLD.SALARY, USER, SYSDATE
    );
END;
/
```

5. Transparent Audit Trigger (AFTER UPDATE OR DELETE)

```
CREATE OR REPLACE TRIGGER cust_mstr_audit
AFTER UPDATE OR DELETE ON CUST_MSTR
FOR EACH ROW
BEGIN
    IF UPDATING THEN
        INSERT INTO CUST_MSTR_AUDIT VALUES (
            :OLD.CUST_ID, :OLD.CUST_NAME,
:OLD.ADDRESS,
            :OLD.PHONE, 'UPDATE', USER,
SYSDATE
        );
    ELSIF DELETING THEN
        INSERT INTO CUST_MSTR_AUDIT VALUES (
            :OLD.CUST_ID, :OLD.CUST_NAME,
:OLD.ADDRESS,
            :OLD.PHONE, 'DELETE', USER,
SYSDATE
        );
    END IF;
END;
/
```

Quick Trigger Reference Table for Viva

Trigger Keyword	Purpose	Example Usage in Lab
`:NEW.column_name`	Accesses new value being inserted or	`:NEW.EMP_NAME :=`